(continued)

2. Create an .env file in the project root directory and add your Google API key:

```
GEMINI_API_KEY=your_actual_gemini_api_key_here
```

3. Execute the desired script using Node.js. For example, to run the embeddings example, use the following command:

```
$ node embeddings.google.js
```

You can run each script individually to explore different functionalities and implementations discussed in this chapter.

This will print a series of numbers in the console representing the embedding of a text:

```
$ node embeddings.js
[
  -0.020836005,  -0.016921125, -0.004506664,   -0.05085442,   -0.02597347,
   0.029602215,   0.029992402,  0.029107977,   0.010593587,  -0.024360698,
  -0.009377503,  0.0066461912, -0.026077522,   0.034726676,  -0.016582962,
...
```

4. If you are using Open AI, there is another file in the same folder named embeddings.openai.js. Create an .env file in the project root directory and add your Google API key:

```
OPENAI_API_KEY =your_actual_openai_api_key_here
```

5. Then run the desired script using Node.js. For example, to run the embeddings example, use the following command:

```
$ node embeddings.openai.js
```

5.1.1 What exactly are prompts?

At the heart of LLM interaction lies a key concept: *prompts*. These are basically structured communication patterns designed to derive specific responses from AI models. Prompts follow a carefully structured format and are designed to guide the LLM's processing and output generation.

Phrasing a question and submitting it to the AI model involves a key step called *tokenization*. This process is vital, as it transforms our natural language instructions into a format that LLMs can consume efficiently.

Whether it is text, images, or other media, tokenization converts them into sequences of tokens, and vice versa. The process is similar to how computers process high-level programming languages: just as these languages must be compiled into machine code (binary format), our prompts must be tokenized for LLM consumption.